

Лабораторна робота – Атака на базу даних MySQL

Цілі та задачі

В цій лабораторній роботі ви розглянете файл PCAP з попередньої атаки на базу даних SQL.

Частина 1: Відкриття Wireshark і завантаження файлу PCAP.

Частина 2. Перегляд атаки SQL-ін'єкції.

Частина 3: Атака SQL-ін'єкції продовжується...

Частина 4: Атака SQL-ін'єкції надає системну інформацію.

Частина 5: Атака SQL-ін'єкції та інформаційні таблиці.

Частина 6: Завершення атаки SQL-ін'єкції.

Довідкова інформація / Сценарій

Атаки впровадження SQL-коду (SQL-ін'єкції) дозволяють зловмисникам вводити оператори SQL у веб-сайт та отримувати відповідь від бази даних. Це дозволяє нападникам маніпулювати поточними даними у базі даних, підміняти посвідчення та вчиняти інші зловмисні дії.

Для перегляду попередньої атаки на базу даних SQL був створений файл PCAP. В цій лабораторній роботі ви розглянете атаки на базу даних SQL та надасте відповідь на запропоновані запитання.

Необхідні ресурси

- Віртуальна машина робочої станції CyberOps.

Інструкції

Для аналізу мережного трафіку ви будете використовувати програму Wireshark – поширений аналізатор мережних пакетів. Після запуску Wireshark відкрийте раніше збережені дані з мережі та крок за кроком простежте атаку з впровадженням SQL-коду на базу даних SQL.

Частина 1: Відкриття Wireshark і завантаження файлу PCAP.

Застосунок Wireshark можна відкрити за допомогою різних методів на робочій станції Linux.

- Запустіть віртуальну машину робочої станції CyberOps.
- Натисніть на **Applications > CyberOPS > Wireshark** на робочому столі і перейдіть до програми Wireshark.
- В застосунку Wireshark натисніть **Open** всередині програми у розділі Files.
- Перегляньте каталог `/home/analyst/` та знайдіть **lab.support.files**. В каталозі **lab.support.files** відкрийте файл **SQL_Lab.pcap**.

- е. Файл PCAP відкривається в Wireshark і відображає зібраний мережний трафік. Цей файл збору даних триває протягом 8 хвилин (441 секунда) – тривалість цієї атаки SQL-ін'єкції.

No.	Time	Source	Destination	Protocol	Length	Info
0	0.000000	10.0.2.15	10.0.2.4	HTTP	430	HTTP/1.1 302 Found
7	0.005700	10.0.2.4	10.0.2.15	TCP	66	35614->80 [ACK] Seq=589 Ack=365 Win=30336 Len=0 TSval=45840 TSecr=91
8	0.014383	10.0.2.4	10.0.2.15	HTTP	496	GET /dvwa/index.php HTTP/1.1
9	0.015485	10.0.2.15	10.0.2.4	HTTP	3107	HTTP/1.1 200 OK (text/html)
10	0.015485	10.0.2.4	10.0.2.15	TCP	66	35614->80 [ACK] Seq=1019 Ack=3406 Win=36480 Len=0 TSval=45843 TSecr=91
11	0.066625	10.0.2.4	10.0.2.15	HTTP	429	GET /dvwa/dvwa/css/main.css HTTP/1.1
12	0.070400	10.0.2.15	10.0.2.4	HTTP	1511	HTTP/1.1 200 OK (text/css)
13	174.254430	10.0.2.4	10.0.2.15	HTTP	536	GET /dvwa/vulnerabilities/sqli/?id=1%3D1&Submit=Submit HTTP/1.1
14	174.254581	10.0.2.15	10.0.2.4	TCP	66	80->35638 [ACK] Seq=1 Ack=471 Win=235 Len=0 TSval=82101 TSecr=91
15	174.257989	10.0.2.15	10.0.2.4	HTTP	1861	HTTP/1.1 200 OK (text/html)
16	220.490531	10.0.2.4	10.0.2.15	HTTP	577	GET /dvwa/vulnerabilities/sqli/?id=1%27+or+%270%27%3D%270+6Subr HTTP/1.1
17	220.490637	10.0.2.15	10.0.2.4	TCP	66	80->35640 [ACK] Seq=1 Ack=512 Win=235 Len=0 TSval=93660 TSecr=10
18	220.493085	10.0.2.15	10.0.2.4	HTTP	1918	HTTP/1.1 200 OK (text/html)
19	277.727722	10.0.2.4	10.0.2.15	HTTP	630	GET /dvwa/vulnerabilities/sqli/?id=1%27+or+1%3D1+union+select+ HTTP/1.1
20	277.727871	10.0.2.15	10.0.2.4	TCP	66	80->35642 [ACK] Seq=1 Ack=565 Win=236 Len=0 TSval=107970 TSecr=10
21	277.732200	10.0.2.15	10.0.2.4	HTTP	1955	HTTP/1.1 200 OK (text/html)
22	313.710129	10.0.2.4	10.0.2.15	HTTP	659	GET /dvwa/vulnerabilities/sqli/?id=1%27+or+1%3D1+union+select+ HTTP/1.1
23	313.710277	10.0.2.15	10.0.2.4	TCP	66	80->35644 [ACK] Seq=1 Ack=594 Win=236 Len=0 TSval=116966 TSecr=10
24	313.712414	10.0.2.15	10.0.2.4	HTTP	1954	HTTP/1.1 200 OK (text/html)
25	383.277032	10.0.2.4	10.0.2.15	HTTP	680	GET /dvwa/vulnerabilities/sqli/?id=1%27+or+1%3D1+union+select+ HTTP/1.1
26	383.277811	10.0.2.15	10.0.2.4	TCP	66	80->35666 [ACK] Seq=1 Ack=615 Win=236 Len=0 TSval=134358 TSecr=10
27	383.284289	10.0.2.15	10.0.2.4	HTTP	4068	HTTP/1.1 200 OK (text/html)
28	441.804070	10.0.2.4	10.0.2.15	HTTP	685	GET /dvwa/vulnerabilities/sqli/?id=1%27+or+1%3D1+union+select+ HTTP/1.1
29	441.804427	10.0.2.15	10.0.2.4	TCP	66	80->35668 [ACK] Seq=1 Ack=620 Win=236 Len=0 TSval=148990 TSecr=10
30	441.807206	10.0.2.15	10.0.2.4	HTTP	2091	HTTP/1.1 200 OK (text/html)

Зачитання:

Які дві IP-адреси задіяні в цій атаці з використанням SQL-ін'єкції на основі інформації, що відображається?

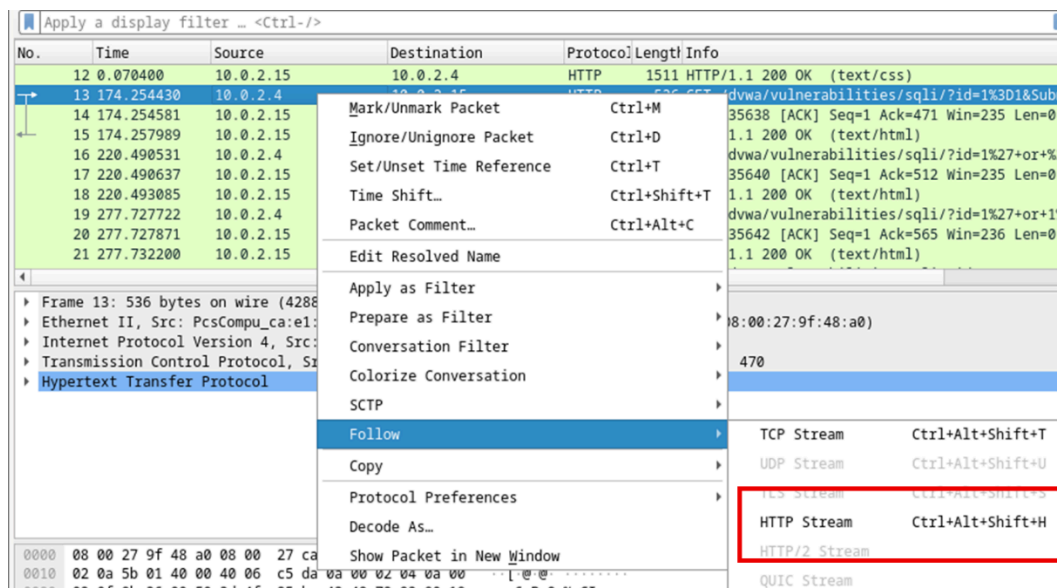
- 10.0.2.4 (атакуючий) та 10.0.2.15 (цільовий веб-сервер).

10.0.2.4 та 10.0.2.15

Частина 2: Перегляд атаки SQL-ін'єкції.

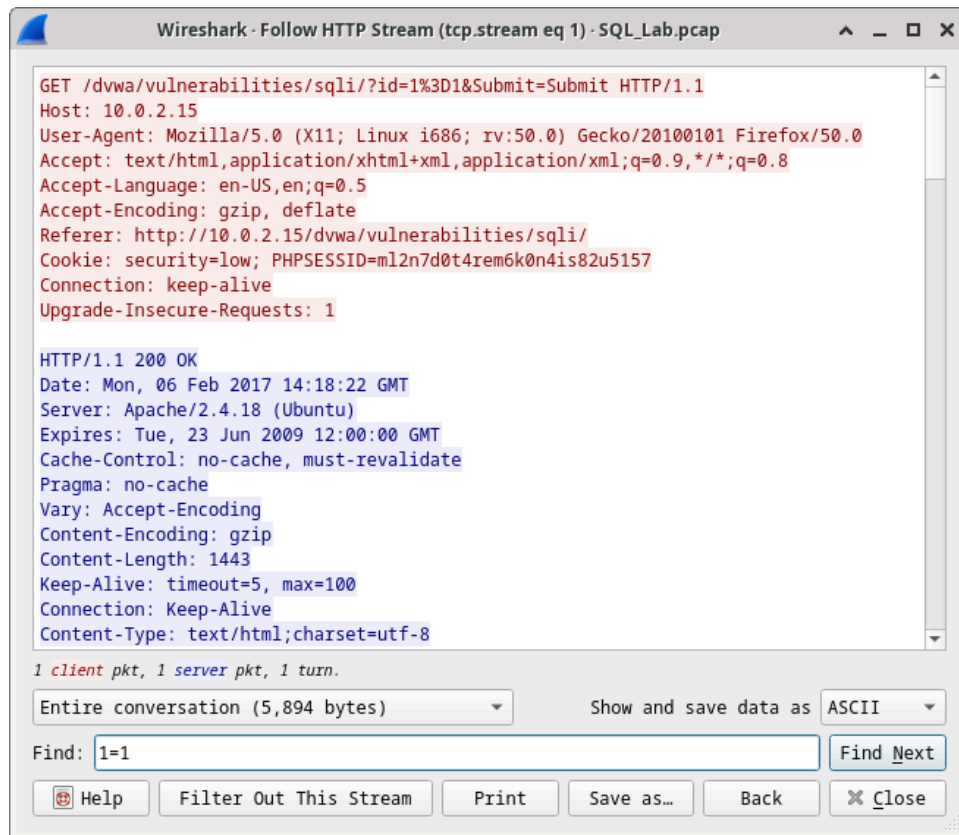
На цьому кроці ви побачите початок атаки.

- а. В даних, зібраних застосунком Wireshark, клацніть правою кнопкою миші по рядку 13 і виберіть **Follow > HTTP Stream**. Рядок 13 був обраний, тому що це HTTP-запит GET. Його дуже зручно використовувати в наступному потоці даних, оскільки прикладні рівні його бачать, що призводить до перевірки запиту на впровадження SQL-коду.

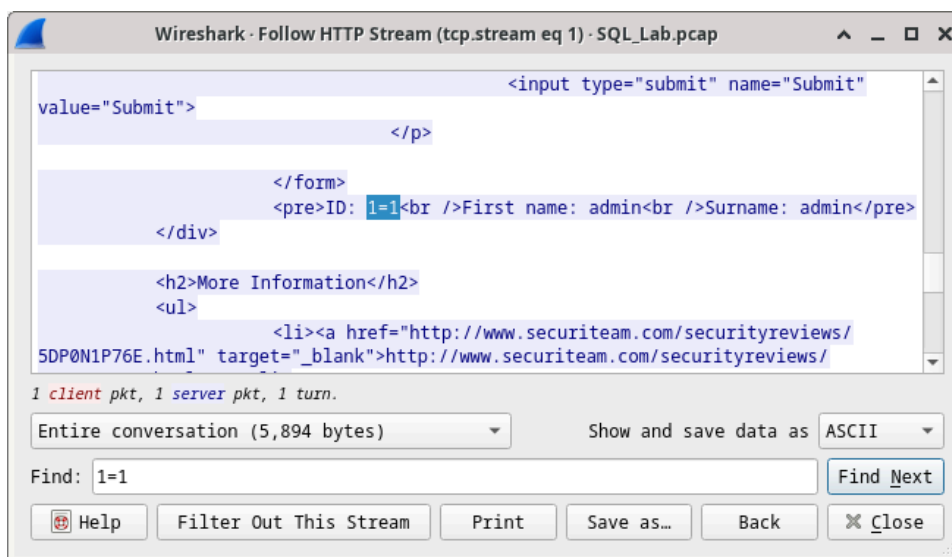


Трафік джерела відображений червоним кольором. З джерела надісланий запит GET на вузол 10.0.2.15. Синім кольором відображено, що пристрій призначення відповідає джерелу.

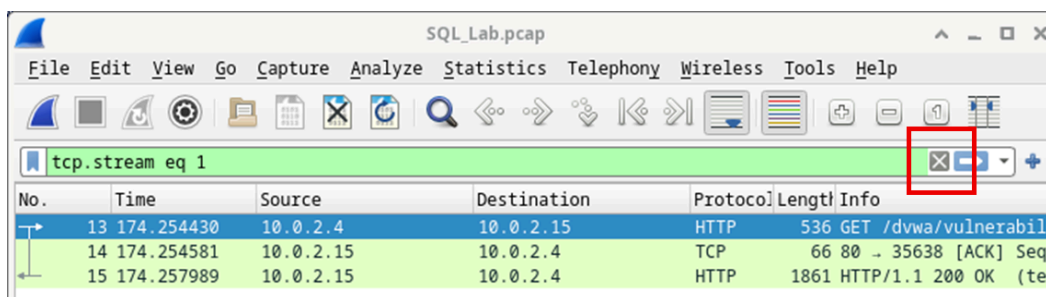
- b. У полі **Find** введіть **1=1**. Натисніть кнопку **Find Next**.



- с. Зловмисник ввів запит (1=1) в поле пошуку UserID на цільовій машині 10.0.2.15, щоб перевірити, чи вразливий застосунок для впровадження SQL-коду. Замість того, щоб програма відповіла повідомленням про помилку входу в систему, вона відповіла записом із бази даних. Зловмисник переконався, що може вводити команди SQL та отримувати відповіді від бази даних. Рядок пошуку 1=1 створює інструкцію SQL, яка завжди має значення true. В даному прикладі не має значення, що буде введене в поле; воно завжди буде мати значення true.



- d. Закрийте вікно Follow HTTP Stream.
- e. Натисніть на кнопку **Clear**, щоб відобразити весь сеанс зв'язку Wireshark.

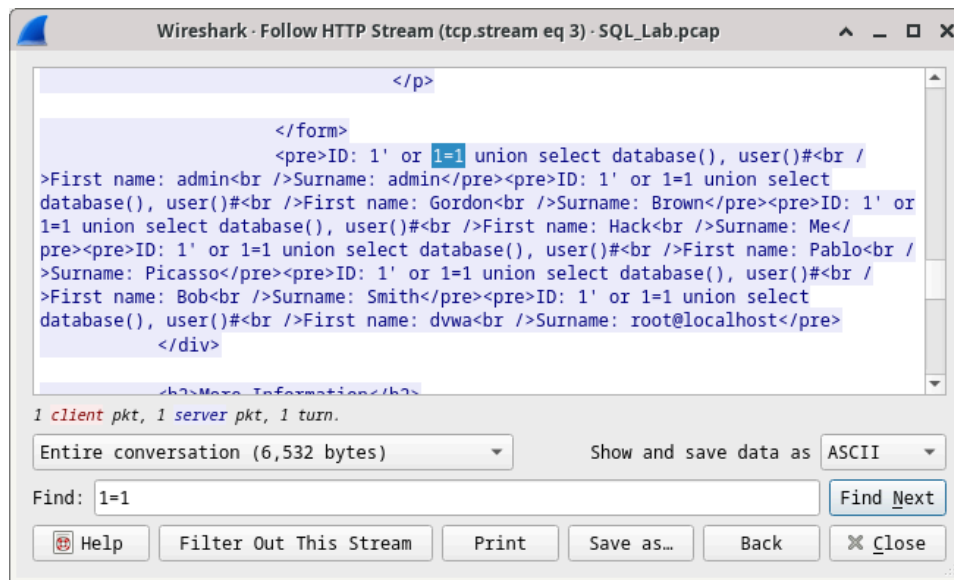


Частина 3: Атака SQL-ін'єкції продовжується...

На цьому етапі ви будете спостерігати за продовженням атаки.

- a. В даних, зібраних застосунком Wireshark, клацніть правою кнопкою миші по рядку 19 і виберіть **Follow > HTTP Stream**.
- b. У полі **Find** введіть **1=1**. Натисніть кнопку **Find Next**.

- с. Зловмисник ввів запит (1' or 1=1 union select database(), user()) в поле пошуку UserID на цільовій машині 10.0.2.15. Замість того, щоб програма відповіла повідомленням про помилку входу в систему, вона відповіла наступною інформацією:



Ім'я бази даних – **dvwa**, а користувач бази даних – **root@localhost**. Також відображаються декілька облікових записів користувачів.

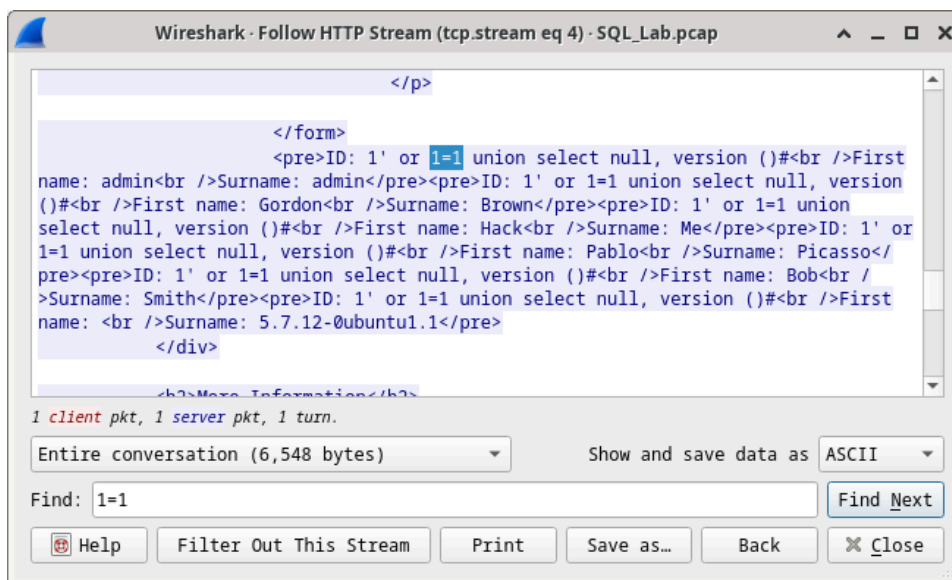
- d. Закрийте вікно Follow HTTP Stream.
- e. Натисніть на кнопку **Clear**, щоб відобразити весь сеанс зв'язку Wireshark.

Частина 4: Атака SQL-ін'єкції надає системну інформацію.

Нападник продовжує і починає шукати більш конкретну інформацію.

- a. В даних, зібраних застосунком Wireshark, клацніть правою кнопкою миші по рядку 22 і виберіть **Follow > HTTP Stream**. Червоним кольором показаний трафік джерела, який надсилає запит GET на вузол 10.0.2.15. Синім кольором відображено, що пристрій призначення відповідає джерелу.
- b. У полі **Find** введіть **1=1**. Натисніть кнопку **Find Next**.

- с. Зловмисник ввів запит (1' or 1=1 union select null, version ())# в поле пошуку UserID цільової машини 10.0.2.15, щоб знайти ідентифікатор версії. Зверніть увагу, що ідентифікатор версії знаходиться в кінці вихідних даних безпосередньо перед </pre>.</div> , який закриває HTML-код.



Запитання:

Яка версія?

- **5.0.51a-3ubuntu5.**

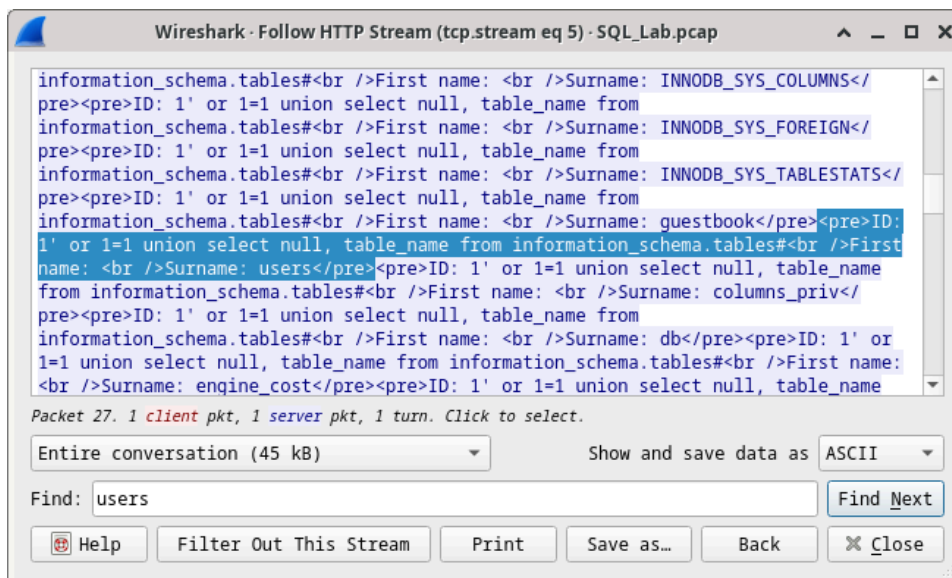
- d. Закрийте вікно Follow HTTP Stream.
- e. Натисніть на кнопку **Clear**, щоб відобразити весь сеанс зв'язку Wireshark.

Частина 5: Атака SQL-ін'єкції та інформаційні таблиці.

Зловмисник знає, що існує велика кількість SQL-таблиць, заповнених інформацією. Зловмисник намагається їх знайти.

- a. В даних, зібраних застосунком Wireshark, клацніть правою кнопкою миші по рядку 25 і виберіть **Follow > HTTP Stream**. Джерело відображено червоним кольором. З джерела надісланий запит GET на вузол 10.0.2.15. Синім кольором відображено, що пристрій призначення відповідає джерелу.
- b. У полі **Find** введіть **users**. Натисніть кнопку **Find Next**.

- с. Зловмисник ввів запит (`1' or 1=1 union select null, table_name from information_schema.tables#`) в поле пошуку UserID на цільовій машині 10.0.2.15 для перегляду всіх таблиць в базі даних. В результаті видається величезна кількість таблиць, оскільки зловмисник вказав «null» без будь-яких додаткових уточнень.



Застереження:

- Чим саме змінена команда (`1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users'`) може допомогти зловмиснику?

*Ця команда дозволяє зловмиснику отримати назви всіх стовпців із конкретної таблиці, яка називається **users**. Знаючи назви стовпців (наприклад, **user**, **password**, **email**), він може побудувати наступний запит для вилучення вже конкретних даних користувачів.*

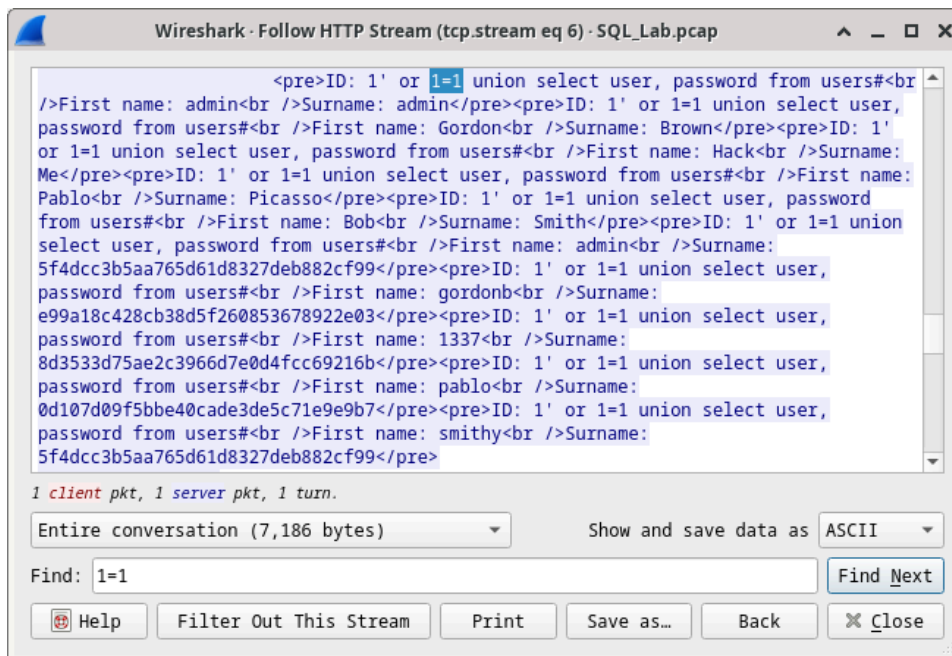
- d. Закрийте вікно Follow HTTP Stream.
e. Натисніть на кнопку **Clear**, щоб відобразити весь сеанс зв'язку Wireshark.

Частина 6: Завершення атаки SQL-ін'єкції.

Атака закінчується найкращим призом з усіх – хеші паролів.

- a. В даних, зібраних застосунком Wireshark, клацніть правою кнопкою миші по рядку 28 і виберіть **Follow > HTTP Stream**. Джерело відображено червоним кольором. З джерела надісланий запит GET на вузол 10.0.2.15. Синім кольором відображено, що пристрій призначення відповідає джерелу.
b. Натисніть **Find** та введіть `1=1`. Здійсніть пошук даного запису. Як тільки текст буде знайдений, натисніть **Cancel** в поле пошуку тексту.

Зловмисник ввів запит (1'or 1=1 union select user, password from users#) в поле пошуку UserID на цільовій машині 10.0.2.15, щоб отримати хеши імен користувачів та паролів!



Запитання:

У якого користувача хеш паролю 8d3533d75ae2c3966d7e0d4fcc69216b?

gordonb

- с. Використовуючи веб-сайт, такий як <https://crackstation.net/>, скопіюйте хеш пароля в зломщик паролів і отримаєте злом.

Запитання:

Що таке простий текстовий пароль?

password

- д. Закрийте вікно Follow HTTP Stream. Закрийте всі відкриті вікна.

Питання для самоперевірки

1. В чому полягає ризик використання мови SQL на платформі?

Основний ризик полягає в тому, що якщо веб-додаток неправильно обробляє дані, введені користувачем, зловмисник може "впровадити" власні SQL-команди. Це дозволяє йому обійти захист, отримати несанкціонований доступ до бази даних, викрасти, змінити або видалити дані.

2. Перегляньте Інтернет і виконайте пошук за запитом «Запобігання атакам з використанням SQL-ін'єкції». Які два способи або кроки можна вжити для запобігання атакам SQL-ін'єкції?

Використання параметризованих запитів (Prepared Statements): Це найефективніший метод. Він розділяє саму SQL-команду від даних, які вводить користувач. Таким чином, дані користувача ніколи не виконуються як частина команди, а розглядаються лише як параметри.

Валідація та санітизація вхідних даних: Завжди перевіряти дані, що надходять від користувача. Наприклад, якщо очікується число, не приймати літери чи символи. Також необхідно "екранувати" спеціальні символи (такі як ' , " , ;), щоб вони не інтерпретувалися як частина SQL-команди.

Кінець документа